

Louisville Weather and Business Risk Monitor Data Warehouse

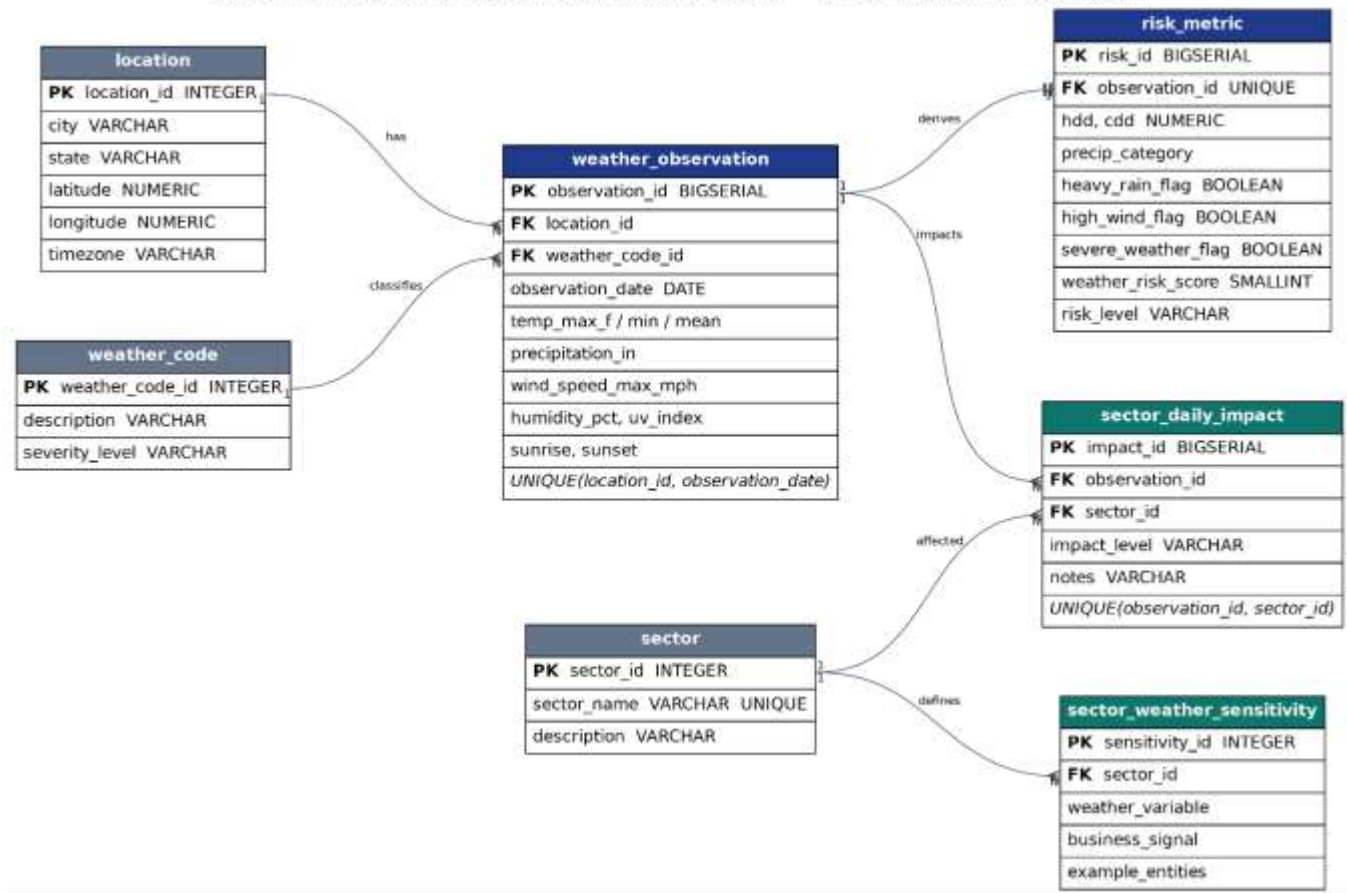
Database Schema Documentation

Developer: Oluwatosin Adelusì

This database stores daily weather forecasts for Louisville, Kentucky, the WMO weather codes that describe each day, the local business sectors exposed to weather, and the automated risk signals produced from the forecast. The system translates raw meteorological data from the Open-Meteo API into business-relevant risk indicators (Heating and Cooling Degree Days, severe-weather flags, and a composite risk score) and loads them into a PostgreSQL database that powers the Plotly Dash. The schema adheres to Third Normal Form (3NF) by separating distinct entities into well-defined tables, minimizing data duplication across location and weather-related records, and using a reference table to capture the weather-to-sector business logic without repeating it on every row.

ER Diagram

Louisville Weather and Business Risk Dashboard — Entity Relationship Diagram



Database Overview

The database contains seven tables:

- Location
- Weather code
- Sector
- Weather observation
- Risk metric
- Sector_weather_sensitivity
- Sector_daily_impact

Entity Relationship Summary

Location: Stores the single monitored geographic location (Louisville, KY) with coordinates and time zone.

Weather Code: Lookup table mapping WMO numeric codes to human-readable descriptions and severity groupings.

Sector: Stores the Louisville business sectors reported on by the dashboard.

Weather Observation: Stores daily weather metrics fetched from the Open-Meteo API for the monitored location.

Risk Metric: Stores the derived degree days, weather flags, and composite risk score for each observation.

Sector_Weather_Sensitivity: Reference table mapping each weather variable to the business signal it drives for a sector.

Sector_Daily_Impact: Records the assessed weather impact level per sector for each forecast day.

Definitions

Location

Stores the single monitored geographic location (Louisville, KY). Centralizing location data here prevents coordinates and time zone from being repeated in every observation row.

Primary Key: location_id

Relationships:

One location can relate to many weather_observation records.

Referenced by weather_observation.location_id

Constraints: UNIQUE (city, state)

Table Structure

Column Name	Data Type	Key	Description
location_id	INTEGER	Primary Key	Auto-generated unique location identifier
city	VARCHAR		City name - Louisville
state	VARCHAR		State abbreviation - KY
latitude	NUMERIC		Decimal latitude coordinate (38.2542)
longitude	NUMERIC		Decimal longitude coordinate (-85.7594)
timezone	VARCHAR		IANA time zone string - America/New_York

Weather_code

Lookup table mapping WMO numeric codes to human-readable descriptions. Separating this from weather_observation prevents the same description from being stored repeatedly.

Primary Key: weather_code_id

Relationships:

One weather_code can describe many weather_observation records.

Referenced by weather_observation.weather_code_id

Table Structure

Column Name	Data Type	Key	Description
weather_code_id	INTEGER	Primary Key	WMO weather interpretation code (e.g. 0 = Clear sky, 61 = Slight rain)
description	VARCHAR		Human-readable label for the weather condition
severity_level	VARCHAR		Coarse severity grouping (normal, moderate, severe)

Sector

Stores the Louisville business sectors used for the dashboard's sector spotlights, such as Utilities, Retail, Logistics, Tourism, and Insurance.

Primary Key: sector_id

Relationships:

One sector can have many sector_weather_sensitivity rules.

One sector can appear in many sector_daily_impact records.

Constraints: UNIQUE (sector_name)

Table Structure

Column Name	Data Type	Key	Description
Sector_id	INTEGER	Primary Key	Auto-generated sector identifier
Sector_name	VARCHAR	Unique	Business sector name - e.g. Utilities, Retail
Description	VARCHAR		Businesses represented by the sector

Weather_observation

Stores aggregated daily weather data fetched from the Open-Meteo API. Each row represents one forecast day for Louisville, KY and is the primary input to the risk-metric decision engine.

Primary Key: observation_id

Foreign Keys:

location_id -> location.location_id

weather_code_id -> weather_code.weather_code_id

Relationships:

Many weather_observation records can reference one location.

Many weather_observation records can reference one weather_code.

One weather_observation drives exactly one risk_metric record.

One weather_observation can produce many sector_daily_impact records.

Constraints: UNIQUE (location_id, observation_date)

Table Structure

Column Name	Data Type	Key	Description
Observation_id	BIGSERIAL	Primary Key	Auto-generated observation record ID
Location_id	INTEGER	Foreign Key	Link to the monitored location
Weather_code_id	INTEGER	Foreign Key	WMO weather condition code for the day
Observation_date	DATE	Unique	Calendar date of the observation
Temp_max_f	NUMERIC		Maximum daily temperature (F)
Temp_min_f	NUMERIC		Minimum daily temperature (F)
Temp_mean_f	NUMERIC		Mean daily temperature (F)
Apparent_temp_f	NUMERIC		Feels-like temperature (F)
Precipitation_in	NUMERIC		Total daily precipitation (inches)
Wind_speed_max_mph	NUMERIC		Maximum wind speed (mph)
Humidity_pct	NUMERIC		Relative humidity (%)
Cloud_cover_pct	NUMERIC		Cloud cover (%)
UV_index	NUMERIC		Maximum UV index
Sunrise	TIME		Sunrise time
sunset	TIME		Sunset time

Risk Metric

Stores the decision-engine output. Each row holds the derived business signals for one weather_observation, computed by comparing the day's metrics against the project's threshold rules in the ETL pipeline.

Primary Key: risk_id

Foreign Keys:

observation_id -> weather_observation.observation_id

Relationships:

Each risk_metric row belongs to exactly one weather_observation (one-to-one).

Constraints: UNIQUE (observation_id)

Table Structure

Column Name	Data Type	Key	Description
risk_id	BIGSERIAL	Primary Key	Auto-generated risk record ID
observation_id	BIGINT	Foreign Key / Unique	The observation these metrics describe
hdd	NUMERIC		Heating Degree Days = $\max(0, 65 - \text{mean F})$
cdd	NUMERIC		Cooling Degree Days = $\max(0, \text{mean F} - 65)$
precip_category	VARCHAR		Dry, Light Rain, Moderate Rain, Heavy Rain
heavy_rain_flag	BOOLEAN		True when precipitation > 1 inch
high_wind_flag	BOOLEAN		True when wind speed > 25 mph
extreme_temp_flag	BOOLEAN		True when mean < 32 F or > 95 F
severe_weather_flag	BOOLEAN		Composite severe-condition flag
weather_risk_score	INT		Composite daily risk score (0-100)
risk_level	VARCHAR		calm, mild, elevated, high, severe

Sector_weather_sensitivity

Reference table capturing the Business Risk Indicator Logic from the project proposal: which weather variable drives which business signal for each sector. Keeping this logic in its own table avoids repeating it on every daily record.

Primary Key: sensitivity_id

Foreign Keys:

sector_id -> sector.sector_id

Relationships:

Many sensitivity rules can belong to one sector.

Constraints: UNIQUE (sector_id, weather_variable)

Table Structure

Column Name	Data Type	Key	Description
sensitivity_id	INTEGER	Primary Key	Auto-generated identifier
sector_id	INTEGER	Foreign Key	Sector the rule applies to

weather_variable	VARCHAR		Driving weather variable (e.g. Mean temperature)
business_signal	VARCHAR		Business interpretation of the variable
example_entities	VARCHAR		Representative Louisville businesses

Sector_daily_impact

Stores the per-sector impact assessment for each forecast day. Each row is derived in the ETL pipeline by applying the sector rules to that day's risk_metric values, and it powers the dashboard's sector spotlight panels.

Primary Key: impact_id

Foreign Keys:

observation_id -> weather_observation.observation_id

sector_id -> sector.sector_id

Relationships:

Many sector_daily_impact records can reference one weather_observation.

Many sector_daily_impact records can reference one sector.

Constraints: UNIQUE (observation_id, sector_id)

Table Structure

Column Name	Data Type	Key	Description
impact_id	BIGSERIAL	Primary Key	Auto-generated identifier
observation_id	INT	Foreign Key	Day the impact applies to
sector_id	INTEGER	Foreign Key	Affected sector
impact_level	VARCHAR		low, moderate, high, severe
notes	VARCHAR		Optional explanation

Cardinality Relationships

Parent Table	Child Table	Relationship Type
location	weather_observation	One-to-Many
weather_code	weather_observation	One-to-Many
weather_observation	risk_metric	One-to-One
sector	sector_weather_sensitivity	One-to-Many
weather_observation	sector_daily_impact	One-to-Many
sector	sector_daily_impact	One-to-Many

Normalization Notes (3NF)

This schema is normalized to approximately Third Normal Form:

- Repeating geographic information is separated into the location table.
- Weather descriptions and severity groupings are separated into weather_code.
- Daily measurements are isolated in weather_observation to avoid redundancy across dates.
- Derived risk values are stored once per day in risk_metric, keyed to their observation.
- The weather-to-sector business logic is separated into sector_weather_sensitivity.
- Daily sector assessments are stored in sector_daily_impact, linking observations and sectors without duplicating underlying data.
- Non-key attributes in each table depend only on their primary key, ensuring no partial or transitive dependencies.

Data Sources

Weather Data Source

Daily weather data is sourced from the Open-Meteo API (<https://api.open-meteo.com/v1/forecast>).

The API provides daily temperatures, precipitation, wind speed, humidity, UV index, weather codes, and sunrise/sunset times.

Weather condition codes follow WMO Weather Interpretation standards and are stored in weather_code.

Reference and Derived Data

location, sector, and sector_weather_sensitivity are loaded from project-maintained CSV files.

risk_metric and sector_daily_impact are derived in the ETL pipeline from weather_observation.

Example Relationship Flow

weather_observation.weather_code_id = 95

weather_code.weather_code_id = 95

Description returned: "Thunderstorm"

The matching risk_metric row for that observation records severe_weather_flag = true and weather_risk_score = 70 (risk_level = high), which then produces sector_daily_impact rows such as Logistics = high and Tourism = severe.

This design prevents duplicate weather descriptions and risk logic from being stored repeatedly across daily records.